

Agile Teams in Large-Scale Distributed Context – Isolated or Connected?

Aivars Šāblis

Blekinge Institute of Technology
Karlskrona
Sweden

aivars.sablis@bth.se

Darja Šmite

Blekinge Institute of Technology
Karlskrona
Sweden

darja.smite@bth.se

ABSTRACT

Background: Agile development heavily relies on effective communication and coordination. Large-scale software development, where many teams are involved in developing often complex projects - even more so. As projects grow more complex, there is a finite limit to the amount of information that one person or a team within that project can possess at a given time. When we have one or two agile teams in a project, achieving good communication and coordination is easy. In fact, agile teams can be self-going. However, when there are many teams, should they be governed and supported or autonomous? Can they work in isolation?

Aim: We aim to understand how isolated or connected are the teams in large-scale software development and answer a number of questions: Who supports the teams in their daily work? How often do teams communicate and coordinate with external contacts? Are supporting roles available?

Method: We use the data from 10 teams from two large-scale distributed development projects. Quantitative data was collected through a survey and qualitative data was obtained from interviews and focus groups. We analyzed teams' knowledge networks using social network analysis techniques.

Results: In this paper, we identify a number of governance roles supporting the teams, and discuss their importance, knowledge exchange frequency and contact availability in the context of large-scale distributed projects.

Conclusion: We conclude that teams interact with a large number of supporting roles grouped into four categories. Although the frequency of interaction differed between the two projects, the amount of interaction was quite high. We thus conclude that agile teams in large-scale development depend on the knowledge and information that resides outside of the team, which means that they are not fully self-going and autonomous. We also found that there are larger differences between networking behavior of teams working in the same project than across the two projects. And finally, our results suggest that distribution and distance introduces barriers to networking across remote locations.

© 2016 Association for Computing Machinery. ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of a national government. As such, the Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only.

XP '16 Workshops, May 24 2016, Edinburgh, Scotland Uk

© 2016 ACM. ISBN 978-1-4503-4134-9/16/05...\$15.00

DOI: <http://dx.doi.org/10.1145/2962695.2962705>

Keywords

Global software development; Offshoring; Insourcing; Social network analysis; Distributed agile; Large-scale agile

1. INTRODUCTION AND BACKGROUND

Today, it is not uncommon that large software companies develop large-scale evolvable systems in a distributed context. These projects often are of increased complexity and size both in terms of the software being developed, as well as the organization developing it. With increasing number of developers, experts and supporting roles the number of project members can reach one hundred people or more. This introduces new challenges and many companies try to increase the effectiveness and efficiency of large-scale software development projects [1].

There are various challenges of large-scale software development projects. Software engineering is knowledge-intensive work [2] and large-scale projects require access to an enormous amount of knowledge and skills [11]. Coordination in large-scale software projects consisting of two or more teams requires participation in forums and forums of forums [3]. Large-scale agile software development projects create additional layer of challenges as agile relies on intensive communication and coordination [4]. However, there is a finite limit to the amount of information one person or a team can possess at a given time [6]. When development tasks may require more information than one person or a team can hold, a growing need for communication and coordination with other teams and supporting roles emerges. At the same time, the large scale of such projects makes it also quite challenging to steer and monitor the progress of the many teams and their interdependencies. This puts new demands on the large-scale agile project governance, which shall comply with the agile philosophy by increasing team effectiveness in the bottom-up “empower and reflect” way, and at the same time govern the large project through top-down “command and control” practices.

Both, expertise knowledge and governance-related information can be exchanged in the project network, which can be referred to as a knowledge network. Knowledge network holds the resources in terms of *know-who* and *know-where* [7]. A software team need to rely on expertise coordination, which requires knowing where expertise is located, knowing where expertise is needed, and integrating the right expertise wherever and whenever it's needed [8], however it can be difficult to exchange knowledge with them [9]. At the same time, the large scale puts also certain demands on

project governance; organizations shall put effort into facilitating team coordination, supplying them with domain knowledge and spreading awareness of the ways of working to ensure product quality and customer satisfaction.

2. METHODOLOGY

The work reported in this workshop paper is a part of a larger research project aiming at improving distributed development in an industrial context. The research is carried out in the company, which for confidentiality reasons remains anonymous. Herein we report the results of the survey conducted in two large-scale distributed projects of the same organization – Project A and Project B with onshore and offshore sites involved in both projects. Our focus is on understanding the different governance roles supporting agile teams in large-scale projects.

2.1 Data Collection

2.1.1 Survey

To collect knowledge exchange data, we conducted a survey. In the survey we asked participants to answer questions about whom they contact to transfer or receive relevant knowledge for their work. We aimed to collect systematic data on whom teams exchange product, process and work related knowledge.

We report the results from surveying 10 teams from two large-scale projects. In Project A, the survey was web-based for the onshore site and handed on the paper printouts for the offshore site. In Project B, the survey was handed on the paper printouts for the onshore site and was web-based for the offshore site. When reporting their contacts, participants were given to fill in as many responses as they felt needed in a “free-recall” format. Participants in paper-based surveys were given around 30 minutes. Participants in the web-based survey were given two days. Longer time was required due to the need of the participants to allocate time in their daily work schedule. We completed the list of recalled contacts and clarified these with company representatives to refine the names, roles in the company and location.

In total members of 11 teams participated in the survey for a total of 65 participants. Actual team sizes ranged from 5 to 9. Not all members of all teams completed the survey, so we included teams with three or more respondents, resulting in a usable sample of 63 members and 10 teams, as follows.

Project A: In total, 28 people from 4 teams completed the questionnaire. The response rate for the 4 teams in total was 97% and for individual teams did not fall below 88%.

Project B: In total, 37 people from 7 teams completed the questionnaire. However, one of the teams was removed from the analysis due to a very low response rate (33%). The response rate for the remaining 6 teams in total was 90% and for individual teams did not fall below 71%. The final sample consisted of 35 project members.

In the survey we elicited the scores for the frequency of receiving knowledge and information from others, based on a 5-point Likert scale (from 1 - never to 5 - daily). We also asked our respondents to state whether supporting roles are perceived as available, based on a 5-point Likert scale (from 1 – strongly disagree to 5 – strongly agree). Our survey is a partial replication of a survey conducted by Manteli et al. [10]. However, we extend it by obtaining a directed knowledge network and acquiring detailed information about each knowledge-sharing relation. In contrast to Manteli et al., who

applied the survey on the project and unit level, we trace our observations to the team level.

2.1.2 Interviews and focus groups

In this paper we also report the project and team context information elicited from additional interviews and focus groups. Interviews with selected governance roles and experts, as well as focus groups with selected development teams were organized in both projects. The focus groups followed a structured agenda where part of the time was dedicated to acquire the information about the presence of the skills needed for solving team’s tasks, teamwork practices, interaction with other teams, roles and communities, usefulness of the externalized knowledge, teams’ reliance on different types of knowledge and skills in their daily work. In Project A we had one focus group with both Swedish teams and Project B we had 6 focus groups (one for each team).

The interviews were semi-structured, and aimed at eliciting the context information about the projects. We conducted 4 interviews in Project A with one technical architect, one with Line manager and two with product owners, while in Project B we conducted 15 interviews – eight interviews in Sweden with an agile coach, a planning manager, a product manager, two technical experts (TARs), release manager, two design owners, system owner, and the system architect, and seven interviews in China with an agile coach, a team responsible manager, two operational product owners, a TAR, previous system owner and a node architect.

2.2 Data Analysis

Data analysis was performed in iterations, and for multiple purposes. In this paper, we first provide detailed statistics of ten different teams of two projects, and connect our observations with the characteristics of the teams and project governance context obtained through interviews and focus groups.

The survey data was analyzed using basic statistics and visualized in a table format using heat maps. The frequency of receiving knowledge and information from the network is calculated based on the values provided by the respondents. Since Technical experts/Architects in the teams reported a large share of the knowledge exchange connections but not all teams had these roles in the team, we excluded them from respective scores. Finally, for interpretation of our statistics, we use our findings from the interviews and focus groups to explain the context and causes.

3. RESULTS

In this section, we present our results regarding the different supporting roles in teams’ knowledge networks in the large-scale agile distributed software projects.

3.1 Project and Process Overview

In both cases projects are large-scale distributed development that can be characterized as an environment in which *nobody knows everything*. Thus, teams working on complex or unfamiliar tasks are required to coordinate within the team, with other teams, and with supporting roles in the rest of the organization.

Projects. The two studied projects are both development of generic software products offered to an open market and are complex compound systems with customized versions. The company has worked in agile development for almost 10 years and at the moment of investigation agile practices have become their way of working.

Both projects are sub-systems that have multiple components and interfaces with other sub-systems. The sub-systems are very complex and contain several millions lines of code.

Project A: Source code compromises both legacy code and new code. Project teams are assigned to bug fixing and feature work at the same time as they work on improvements and attend meetings. Also in some teams there are a lot of part-timers, members having additional responsibilities outside the team, which increases task switching, limits available resources for feature work and creates problems to teams working well.

Project B: Teams are assigned to feature development and bug fixing, which is a rotating task. Therefore, teams usually spend dedicated time for only feature development or bug fixing. There are very few part-timers in this project.

Teams. Development in both projects is organized in cross-functional feature teams (XFT) responsible for a feature starting from high level description of a feature until its release to the customer. All teams have all core competences needed for the development and release of a feature. In both projects, the XFT includes system managers, developers, function testers, and system testers. In addition, each XFT has a scrum master. In both projects, the teams consist of five - nine team members. Team members might have several roles in their team. For example, there are some teams with part-time team members having the role of a technical expert available for consultation services to other teams. The teams are usually situated in an open plan office.

Table I. Total contacts and connections for teams

	Team	Network contacts			Network connections		
		total	remote	local	total	remote	local
Project A	A1	17	2	15	19	2	17
	A2	22	3	19	25	3	22
	A3	7	0	7	18	0	18
	A4	46	8	38	61	8	53
Project B	B1	48	12	36	69	10(8*)	59
	B2	11	2	9	25	4(0*)	41
	B3	26	2	24	35	2(0*)	32
	B4	21	4	17	42	4(1*)	38
	B5	29	7	22	40	10(5*)	30
	B6	25	3	22	31	3(0*)	28

*excluding Technical experts/architects

Supporting roles. In Project A we have surveyed 2 teams in the onshore site and 2 teams in the offshore site. The respondents from these teams in total identified 38 persons interacting with the teams (24 persons from the onshore site and 14 persons from the offshore site), with 6-8 supporting roles per team (exception was team A4, in which team leader had very large communication network with 11 roles). In Project B we surveyed 3 teams in the onshore site and 3 teams in the offshore site. Participants from these teams identified 83 persons (40 persons in the onshore site and 43 in the offshore site), or 9-13 supporting roles per team. The exception was team B4, which was very immature team (average experience in team was less than one year), and had only 6 contacts. In Project A, all teams had generally smaller networks then in Project B, both with respect to the number of contacts and connections reported in the survey, see Table II.

Both projects are large-scale agile distributed development projects and therefore relies on multiple additional roles to support the teams. Based on the survey responses we learned that respondents in Project A identified 11 types of supporting roles and respondents in Project B identified 12 types of supporting roles. Supporting roles in both projects include agile coaches, integration leaders, line managers, product owners, program managers, scrum

masters, system architects, system managers, technical experts/architects (TA/A) and test support specialists (TestSpec). In addition, in Project A team members identified technical test coordinator role (TTC) and in Project B – configuration managers (CM), and development support specialists (DevSpec). Team members in the both projects also communicated with developers (Dev) and testers from other teams. Since the projects are very large, some of the supporting roles required the work of several full time employees. Some of these roles also had local representatives in each of the development location.

Table III. Contacts in each supporting role

		Project A		Project B	
		Onshore	Offshore	Onshore	Offshore
Line Management	Agile Coach	1	0	1	0
	Line Manager	1	1	2	2
	Program Manager	1	2	0	1
	Scrum Master	0	2	1	7
Product Management	Solution Manager	0	0	0	1
	System Architect	1	1	1	1
	Technical Expert/Architect	8	2	7	6
Feature Development Support	Documentation Writer	0	0	2	0
	Product Owner	2	2	2	2
	System Manager	7	2	10	6
	Technical Test Coordinator	1	2	0	0
Development Support	Configuration Manager	0	0	1	2
	Development Support Specialist	0	0	5	9
	Integration Leader	1	0	2	0
	Test Support Specialist	3	0	6	6
Other teams	Developers	15	18	11	15
	Tester	0	18	7	2

Based on the purpose of support, we grouped the identified roles into four major categories – Project Management, Product Management, Feature Development Support and Development Support. Representatives from the first group, Project Management, fulfill the functions of project governance and control, while those from the second group govern product requirements, vision and scope, and fulfill the quality control functions (Technical experts/architects). In the third group, Feature development support, we have united roles that sit closely with the teams and support their development task execution, while roles from the fourth group, Development Support roles, coordinate the outcome of all teams in the line, e.g. by fulfilling testing and integration tasks. Evidently, these are not all possible roles involved in the development. Our intention here is to demonstrate how many roles do teams interact with directly. See the summary in Table II, where we report the number of people in different supporting roles under each category and the division between onshore and offshore locations.

3.2 Communication Frequency

One of our main areas of interest is related to team autonomy in the large-scale agile software development. In other words, how dependent are teams on the knowledge external to the team. This can be demonstrated by looking into amount and frequency of receiving the knowledge from others. Our results are visualized as a heat map (see Table III), where we show the frequency of receiving knowledge and information from others.

We found that for all supporting roles the scores in Project B are lower than in Project A. Based on the interviews and focus groups we learned that teams in Project A have a large number of tasks,

Table III. Dependency from the knowledge and information from supporting roles

			Project A				Project B					
	Proje	Roles	A1	A2	A3	A4	B1	B2	B3	B4	B5	B6
Line Management	A/B	Agile Coach (AC)	3,8	3,7	-	-	2,0	2,0	-	-	-	-
	A/B	Line Manager (LMng)	-	-	3,8	3,3	2,1	2,3	2,0	1,7	1,6	1,5
	A/B	Program Manager (PMng)	-	-	-	4,1	-	-	-	-	-	3,0
	A/B	Scrum Master (SM)	-	-	4,0	4,0	1,5	1,5	-	1,8	-	2,3
Product Management	B	Solution Manager (SMng)	-	-	-	-	2,0	-	-	-	-	-
	A/B	System Architect (SA)	3,5	4,0	-	-	1,5	-	-	1,5	1,3	0,8
	A/B	Technical Expert / Architect (TE/A) - local	2,6	2,9	2,9	3,6	1,4	1,9	2,1	1,8	1,4	1,8
	A/B	TE/A - remote	4,0	-	-	-	1,3	4,0	3,0	1,6	1,2	1,0
Feature Development	B	Documentation Writer (DocWri)	-	-	-	-	2,3	2,9	-	-	-	-
	A/B	Product Owner (PO)	-	3,5	4,0	3,9	2,8	3,0	3,0	-	-	2,5
	A/B	System Manager (SysMng)	3,0	2,8	4,5	3,9	1,6	1,5	1,8	2,6	2,8	1,5
	A	Technical Test Coordinator (TTC)	-	-	3,5	4,0	-	-	-	-	-	-
Development Support	B	Configuration Manager (CM)	-	-	-	-	1,5	-	-	-	1,5	0,8
	B	Development Support Specialist (DevSpec) - local	-	-	-	-	2,4	1,1	1,5	-	1,9	1,7
	B	DevSpec - remote	-	-	-	-	2,5	-	-	-	1,5	1,5
	A/B	Integration Leader (IL)	-	-	-	4,5	2,7	-	1,5	-	-	3,0
	A/B	Test Support Specialist (TestSpec)	3,0	4,0	-	3,3	1,4	1,5	2,0	-	2,3	1,0
Other teams	A/B	Developers (Dev)	2,8	3,1	3,5	3,8	1,8	1,7	1,6	2,5	2,0	1,9
	A/B	Tester	-	-	-	3,9	2,2	1,8	1,1	-	2,0	1,0

and task switching, and therefore a lot of communication related to all the tasks on a daily basis. This could be one explanation for why the frequency of communication in Project A is high. In Project B, teams are less interrupted during feature work and may therefore need to interact less with others. When analyzing the type of knowledge flows, we further learned that in Project A teams receive the knowledge and information related to work coordination, while in Project B teams primarily scout for product-related knowledge. Another interesting observation is that teams in Project B communicate with more roles (see Table II) and therefore might have less time to communicate with each role as much as teams in Project A.

3.3 Availability of supporting roles

We further explore the availability of supporting roles as perceived by the team members. As not all teams had Technical

experts/Architects in the team, we excluded the mutual ties between the technical experts/Architects from respective team scores.

From Table IV we can clearly see that availability of contacts in the teams' networks is not a problem. Overall in the both projects, team members perceive that supporting roles are available for consultation. A few exceptions exist. In Project A and B, a few teams assigned lower scores to the Test support specialists when comparing to other roles. In Project B, lower scores were also given to remote Technical Experts/Architects, remote Development Support Specialists. The perceived poor availability of remote contacts highlights the challenges of working in a distributed environment. Notably, locally Technical Experts/Architects were seen as available, despite their heavy workload that was reported by the teams in the focus groups.

Table IVV. Availability of supporting roles

			Project A				Project B					
	Project	Roles	A1	A2	A3	A4	B1	B2	B3	B4	B5	B6
Project Management	A/B	Agile Coach (AC)	3,0	4,3	-	-	4,5	5,0	-	-	-	-
	A/B	Line Manager (LMng)	-	-	4,7	4,3	4,3	4,5	4,0	4,0	4,5	4,7
	A/B	Program Manager (PMng)	-	-	-	4,3	-	-	-	-	-	5,0
	A/B	Scrum Master (SM)	-	-	4,0	4,0	4,0	5,0	-	5,0	-	5,0
Product Management	B	Solution Manager (SMng)	-	-	-	-	4,0	-	-	-	-	-
	A/B	System Architect (SA)	4,0	4,0	-	-	5,0	-	-	5,0	3,8	4,0
	A/B	Technical Expert / Architect (TE/A) - local	3,8	4,5	4,5	4,5	4,7	4,4	4,1	4,4	4,0	4,5
	A/B	TE/A - remote	4,0	-	-	-	3,5	2,0	4,0	2,5	2,3	3,4
Feature Development Support	B	Documentation Writer (DocWri)	-	-	-	-	4,0	4,8	-	-	-	-
	A/B	Product Owner (PO)	-	4,0	4,2	4,0	4,3	4,0	4,0	-	-	4,0
	A/B	System Manager (SysMng)	3,7	3,5	5,0	4,0	3,7	4,3	4,3	4,9	4,7	3,7
	A	Technical Test Coordinator (TTC)	-	-	5,0	4,0	-	-	-	-	-	-
Development Support	B	Configuration Manager (CM)	-	-	-	-	5,0	-	-	-	5,0	4,3
	B	Development Support Specialist (DevSpec) - local	-	-	-	-	4,6	4,0	3,5	-	4,8	4,0
	B	DevSpec - remote	-	-	-	-	3,8	-	-	-	3,0	3,0
	A/B	Integration Leader (IL)	-	-	-	4,0	4,0	-	4,0	-	-	4,0
	A/B	Test Support Specialist (TestSpec)	2,0	4,0	-	4,5	3,7	3,0	4,0	-	2,8	4,0

Other teams	A/B	Developers (Dev)	4,2	4,4	5,0	4,1	-	4,3	4,4	4,8	5,0	4,4
	A/B	Tester	-	-	-	4,2	4,7	4,0	4,8	-	4,0	5,0

4. CONCLUSIONS AND FUTURE WORK

In this workshop paper, we report our results from analyzing a survey that focused on understanding how much do teams communicate and coordinate externally in large-scale distributed agile projects. Our results show that teams interact with a large number of supporting roles grouped into four categories, and depend on the knowledge and information that resides outside of the team. We also learned that frequency of receiving the knowledge and information among the two projects varied. When comparing the network size and amount of incoming knowledge connections in two projects we can see that there are larger differences between the teams in the same project, than between the projects. Although teams in Project A had on average slightly bigger networks, these seem to depend more on the characteristics of the team and the work they perform. However, the frequency of receiving the information differed greatly among the two projects. This might be due to different approaches in task allocation, the need for process governance and coordination, and availability of the product knowledge in the team. Interestingly, availability of the same contacts was perceived differently by the different teams. In general, we conclude that contacts are available to teams, although remote experts were perceived unavailable. This indicates that distance creates barriers to knowledge exchange and managers should take extra care, when organizing roles and responsibilities across distributed sites.

The number of contacts steering the development in each of the studied projects differed, and so did the frequency of interaction. We also learned that some roles employ several full time employees, while others might be carried out by a single individual and some require only part of the time; some roles are presented in each development location locally, while others only in the central development locations. Unfortunately, the survey data and our insights from the focus groups and interviews do not allow to fully portray the large-scale project governance, since the focus of our initial investigation was on knowledge transfer in the network of teams. Thus, in our future work will elicit the information from the supporting roles.

The results reported here have generated interest in looking more closely into governance issues in large-scale software projects. Our findings suggest that teams in large-scale projects are not fully self-going. The supporting roles resemble signs of plan-driven, hierarchical environment. Our future work will thus focus on investigating the trade offs between the need for coordination and control versus the teams' need for autonomy.

When it comes to distribution, we found very few cross-site connections, mostly mandatory coordination with formal experts, who had a governance role. Our future work will thus focus on understanding the cross-site governance perspective, and whether the lack of interaction across site does not hinder some of the coordination and control needs.

In summary, we would like to answer the following questions:

- Which roles empower the teams in large-scale distributed agile projects? Do the opinions differ across locations?
- Which roles are needed to control the development in large-scale distributed agile projects? Does project control hinder team's autonomy? Do experiences differ across locations?
- Which roles are needed locally and which can operate remotely? Are these needs location-specific or shared?

5. REFERENCES

- [1] J. D. Herbsleb and A. Mockus, "An empirical study of speed and communication in globally distributed software development," *IEEE Transactions on Software Engineering*, 29(6): 481–494, 2003.
- [2] D. Šmite, C. Wohlin, T. Gorschek, and R. Feldt, "Empirical evidence in global software engineering: a systematic review," *Journal of Empirical Software Engineering*, 15(1): 91–118, 2010.
- [3] T. Dingsøyr, T. E. Fægri, and J. Itkonen. "What is Large in Large-Scale? A Taxonomy of Scale for Agile Software Development." *Product-Focused Software Process Improvement*. Springer, 2014. 273-276.
- [4] T. Dybå, and T. Dingsøyr. "What do we know about agile software development?" *IEEE Software*, 26(5): 6-9, 2009.
- [5] T. Dingsøyr, and N. B. Moe. "Research challenges in large-scale agile software development." *ACM SIGSOFT Software Engineering Notes* 38.5: 38-39, 2013.
- [6] J. T. Garner, "It's not what you know: A transactive memory analysis of knowledge networks at NASA." *Journal of technical writing and communication* 36(4): 329-351, 2006.
- [7] I. Rus, and M. Lindvall. "Guest editors' introduction: Knowledge management in software engineering." *IEEE Software* 19(3): 26-38, 2002.
- [8] S. Faraj, and L. Sproull. "Coordinating expertise in software development teams." *Management science journal*, 46(12): 1554-1568, 2000.
- [9] N. B. Moe, D. Šmite, A. Šāblis, A. L. Börjesson, and P. Andréasson. "Networking in a large-scale distributed agile project." *Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. ACM, 2014.
- [10] C. Manteli, B. van den Hooff, and H. van Vliet, "The Effect of Governance on Global Software Development: An Empirical Research in Transactive Memory Systems". *Information and Software Technology*, 56(10): 1309-1321, 2014.
- [11] V.T. Rajlich, and K. H. Bennett, "A staged model for the software life cycle." *Computer* 33(7): 66-71, 2000.